

准循环扭McEliece密钥封装机制 (QCTM)

算法文本

商用密码方案征集提交版

算法类型：公钥密码算法，密钥封装机制（KEM）

算法名称：准循环扭McEliece密钥封装机制（QCTM）

参数实例：QCTM128、QCTM256、QCTM512

提交人、联系人、知识产权声明和签字信息：见同一提交包中的《算法基本信息》、《知识产权所有人声明》和《参考实现和优化实现所有人声明》

实现材料：见同一提交包中的Implementations/Reference_Implementation、Test_Vectors和Supporting_Documentation

2026年6月

摘要

本文给出准循环扭McEliece密钥封装机制 (QCTM) 的算法规范。该方案采用Niederreiter 型校验子密文, 底层码族由带仿射平移的准循环扭Goppa码的结构给出。公钥矩阵不是全局纯准循环系统型矩阵, 而是由局部准循环主体和普通尾行组成的混合系统型矩阵; 公钥保存为 $(\psi(T_1), T_2)$, 其中 T_1 以分块循环形式压缩, T_2 逐行保存; 译码采用扭Goppa码的译码算法。本文中的数学定义、矩阵生成、密钥生成、封装和解封装均以提交实现采用的矩阵构造为准。

目录

1	引言	4
1.1	动机	4
1.2	方案类型	4
2	预备知识	4
2.1	符号	4
2.2	Goppa 码、扭Goppa 码与准循环性	4
3	参数	5
3.1	参数要求	5
3.2	选择参数与多项式	5
3.2.1	参数集QCTM128	6
3.2.2	参数集QCTM256	6
3.2.3	参数集QCTM512	6
3.3	公私钥和密文字节数	6
4	单向函数	7
4.1	Goppa 多项式生成	7
4.2	域元素生成	8
4.3	定义集排序生成	9
4.4	矩阵生成	10
4.5	固定重量向量生成	12
4.6	编码	12
4.7	译码	13
5	密钥封装机制	13
5.1	密钥生成	13
5.2	封装	14
5.3	解封装	14
6	安全性考虑	14
6.1	底层困难问题	14
6.2	实际安全性测算依据	16
6.3	结构攻击	18
6.4	解封装失败分析	19
7	参考和优化实现	19
7.1	不可约多项式 $f(z)$ 和 $F(y)$ 的实例化	19
7.2	矩阵验证	20

7.3 参考实现	20
7.4 优化实现	20
8 性能与尺寸	20
8.1 算法性能（千时钟周期数）	20
8.2 参数尺寸（字节数）	21
8.3 已知答案测试	21
9 创新性声明	21
10 结论	21

1 引言

1.1 动机

经典McEliece 密码系统[30]安全性强但是公钥体积过大；BIG QUAKE利用Goppa码准循环结构压缩公钥，体积变小但是带来了潜在的攻击风险；所以我们采用了准循环和尾项结构，相对于BIG QUAKE方案提高了安全性；相对于经典McEliece 密码系统在同等安全级别下公钥尺寸显著降低。

QCTM 的目标是在保持Niederreiter 型密文[32, 38]和扭Goppa 译码结构的前提下，使用如下混合公钥形式：

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}_{u \times n}, \quad u = m(t-1) + 1 \equiv r_0 \pmod{\ell},$$

其中 T_1 是 ℓ -分块循环矩阵， T_2 是 r_0 行普通二元矩阵。公钥只压缩 T_1 ， T_2 保持不动。

1.2 方案类型

本方案是密钥封装机制。封装阶段采样固定重量错误向量 e ，计算公开校验子 $C = He^T$ ，并从 (e, C) 派生会话密钥。解封阶段使用私钥中保存的扭Goppa 结构译码恢复 e ，成功时输出相同会话密钥，失败时使用私钥中的随机失败分支向量 s 输出伪随机密钥。

2 预备知识

2.1 符号

本文使用如下符号。

符号	含义
q	2^m
m	二元扩域 $\mathbb{F}_q/\mathbb{F}_2$ 的扩张次数
ℓ	奇素数，满足 $\ell \mid q-1$
ζ	$\mathbb{F}_q$ 中阶为 ℓ 的元素
n	码长，满足 $\ell \mid n$
t_0	奇数
t	Goppa 多项式次数， $t = \ell t_0$
k	扭Goppa码维数 $k = n - m(t-1) - 1$
w	封装错误重量， $w = (t-1)/2$
η	KeyGen 初始随机选取的 $\mathbb{F}_q^*$ 中元素
g	$\mathbb{F}_q[x]$ 中次数为 t 的首一不可约多项式
L	扭Goppa 码定义集 $\{\alpha_0, \dots, \alpha_{n-1}\}$
Γ	二元扭Goppa 码 $\Gamma(g, L, \eta)$
$\mathbf{H}$	密钥派生所用杂凑函数
$\mathbf{G}$	随机种子扩展所用伪随机生成器

向量均按行写出，校验子计算采用列向量 e^T 。二元矩阵的行列下标在算法描述中默认从0 开始；若用“第 $1, \ell+1, 2\ell+1, \dots$ 行”表述，则表示同一集合的1 起始编号。

2.2 Goppa 码、扭Goppa 码与准循环性

定义2.1 (二元Goppa 码). [20, 21, 23] 设 $g \in \mathbb{F}_q[x]$ 是次数为 t 的首一不可约多项式， $L = \{\alpha_0, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_q$ ，且 $g(\alpha_j) \neq 0$ 对所有 $0 \leq j < n$ 成立。二元Goppa 码定义为

$$\Gamma(L, g) = \left\{ c = (c_0, \dots, c_{n-1}) \in \mathbb{F}_2^n : \sum_{j=0}^{n-1} \frac{c_j}{x - \alpha_j} \equiv 0 \pmod{g(x)} \right\}.$$

定义2.2 (二元扭Goppa 码). 设 $g \in \mathbb{F}_q[x]$ 、 $L = \{\alpha_0, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_q$ 满足 $g(\alpha_j) \neq 0$ ，并设 $\eta \in \mathbb{F}_q$ 。二

元扭Goppa 码定义为

$$\Gamma(L, g, \eta) = \left\{ c = (c_0, \dots, c_{n-1}) \in \mathbb{F}_2^n : \sum_{j=0}^{n-1} c_j \left(\frac{1}{x - \alpha_j} - \eta \frac{\alpha_j^t}{g(\alpha_j)} \right) \equiv 0 \pmod{g(x)} \right\}.$$

当 $\eta = 0$ 时, $\Gamma(L, g, \eta)$ 即为普通Goppa 码 $\Gamma(L, g)$ 。

定义2.3 (准循环码). 设 $\mathcal{C} \subseteq \mathbb{F}_2^n$ 是长度为 n 的线性码。若 $c = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}$, $\sigma_l(c) = (c_{-l}, c_{1-l}, \dots, c_{n-1-l}) \in \mathcal{C}$, $d = \gcd(l, n)$, 则 $\mathcal{C}$ 是一个 d 阶的准循环码。当 $l = 1$ 时, $\mathcal{C}$ 为循环码。

有关扭Goppa码和准循环码的文献可参考[5, 6, 7, 8, 9, 24, 25, 26, 27]。

定义2.4 (ℓ -分块循环矩阵). 设 ℓ 为正整数, M 为一个矩阵。若 M 可分解为若干个 $\ell \times \ell$ 循环块, 且每个 ℓ -循环块均形如

$$\begin{pmatrix} a_0 & a_1 & \cdots & \cdots & a_{\ell-1} \\ a_{\ell-1} & a_0 & \cdots & \cdots & a_{\ell-2} \\ \vdots & \vdots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ a_1 & a_2 & \cdots & a_{\ell-1} & a_0 \end{pmatrix},$$

则称 M 为 ℓ -分块循环矩阵。

3 参数

3.1 参数要求

首先, QCTM方案使用以下编码定义参数:

1. 正整数 m , 并令 $q = 2^m$ 。
2. 奇素数 ℓ , 满足 $\ell \mid q - 1$ 。
3. 正整数 $n \leq q$, 且 $\ell \mid n$ 。
4. 奇数 t_0 , 满足 $m\ell t_0 < n$, $t = \ell t_0$, 和 $w = (t - 1)/2$ 。
5. ζ 为 $\mathbb{F}_q$ 中的 ℓ 阶元素。
6. $f(z) \in \mathbb{F}_2[z]$ 为次数 m 的首一不可约多项式, 用于表示 $\mathbb{F}_q = \mathbb{F}_2[z]/f(z)$ 。
(具体形式详见第7节。)
7. $F(y) \in \mathbb{F}_q[y]$ 为次数 t_0 的首一不可约多项式, 用于表示 $\mathbb{F}_{q^{t_0}} = \mathbb{F}_q[y]/F(y)$ 。
(具体形式详见第7节。)

其次, QCTM方案使用以下对称密码参数及组件:

1. 正整数 $\lambda \geq 256$ 。(对于QCTM128和QCTM256参数集, 令 $\lambda = 256$; 对于参数集QCTM512, 令 $\lambda = 512$ 。)
2. 密码学杂凑函数 **H**, 输出长度为 λ 。(可以用算法征集平台提供的 **CryptHash** 来实例化。)
3. 伪随机数生成器 **G**, 将长度为 λ 比特串扩展成长度为 $n + \sigma_2(q - 1) + \sigma_1(t_0 + 1) + \lambda$ 的比特串, 其中 $\sigma_1 \geq m$, $\sigma_2 \geq 2m$ 。(可以用算法征集平台提供的 **get_random_number** 来实例化。)

3.2 选择参数与多项式

为满足QCTM 所需拟循环扭曲Goppa 码的结构设计要求, 每一组提交参数实例都需要同时给出其KEM 参数以及两个不可约多项式

$$f(z) \in \mathbb{F}_2[z] \text{ 和 } F(y) \in \mathbb{F}_q[y]$$

其中 $\mathbb{F}_q \triangleq \mathbb{F}_2[z]/(f(z))$ 。综合考虑实现效率, 本文选取如下三组提交参数实例。

3.2.1 参数集QCTM128

该KEM 的参数为 $m = 18$ 、 $n = 10070$ 、 $\ell = 19$ 、 $t_0 = 15$ 、 $t = 285$ 、 $k = 4957$ 、 $w = 142$ 。公钥长度为167,919 字节，私钥长度为23,299 字节，密文长度为640 字节。域多项式选取为 $f(z) = z^{18} + z^3 + 1$ ， $F(y) = y^{15} + y^7 + z^7$ 。

3.2.2 参数集QCTM256

该KEM 的参数为 $m = 18$ 、 $n = 19000$ 、 $\ell = 19$ 、 $t_0 = 27$ 、 $t = 513$ 、 $k = 9783$ 、 $w = 256$ 。公钥长度为595,541 字节，私钥长度为43,905 字节，密文长度为1,153 字节。域多项式选取为 $f(z) = z^{18} + z^7 + 1$ ， $F(y) = y^{27} + z$ 。

3.2.3 参数集QCTM512

该KEM 的参数为 $m = 18$ 、 $n = 38000$ 、 $\ell = 19$ 、 $t_0 = 53$ 、 $t = 1007$ 、 $k = 19891$ 、 $w = 503$ 。公钥长度为2,374,489 字节，私钥长度为87,766 字节，密文长度为2,264 字节。域多项式选取为 $f(z) = z^{18} + z^3 + 1$ ， $F(y) = y^{53} + y^6 + z$ 。

3.3 公私钥和密文字节数

令

$$u = m(t - 1) + 1 \equiv r_0 \pmod{\ell}, \quad 0 \leq r_0 < \ell.$$

系统化后的公开矩阵为

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}_{u \times n},$$

其中 T_1 为 $(u - r_0) \times (n - u + r_0)$ 阶 ℓ -分块循环矩阵， T_2 为 $r_0 \times (n - u + r_0)$ 阶普通二元矩阵。因此公钥负载比特数为

$$\text{pkbits} = \frac{u - r_0}{\ell}(n - u + r_0) + r_0(n - u + r_0),$$

即公钥的字节数为

$$\text{pkbytes} = \left\lceil \frac{1}{8} \left(\frac{m(t - 1) + 1 - r_0}{\ell} (n - (m(t - 1) + 1) + r_0) + r_0(n - (m(t - 1) + 1) + r_0) \right) \right\rceil.$$

私钥的字节数为

$$\text{skbytes} = \left\lceil \frac{1}{8} (n + t)m \right\rceil.$$

密文的字节数为

$$\text{ctbytes} = \left\lceil \frac{1}{8} (m(t - 1) + 1) \right\rceil.$$

注：对于参数集QCTM128、QCTM256 和QCTM512，相应的公钥、私钥及密文尺寸详见第8.2节。

4 单向函数

4.1 Goppa 多项式生成

Algorithm 1: GoppaPolynomial

Input: 比特串 $d_0, d_1, \dots, d_{\sigma_1(t_0+1)-1}$, 长度为 $\sigma_1(t_0+1)$
Output: $\perp$ 或 (η, g) , 其中 $\eta \in \mathbb{F}_q^*$, $g \in \mathbb{F}_q[x]$ 是次数为 $t = \ell t_0$ 的首一不可约多项式

- 1 将输入比特串划分为 $t_0 + 1$ 段, 每段 σ_1 比特, 仅使用每段前 m 个比特;
- 2 **for** $j = 0$ **to** $t_0 - 1$ **do**
- 3 $\beta_j \leftarrow \sum_{i=0}^{m-1} d_{\sigma_1 j + i} z^i \in \mathbb{F}_q$;
- 4 $\eta \leftarrow \sum_{i=0}^{m-1} d_{\sigma_1 t_0 + i} z^i \in \mathbb{F}_q$;
- 5 **if** $\eta = 0$ **then**
- 6 $\perp$ **return** $\perp$;
- 7 $\beta \leftarrow \beta_0 + \beta_1 y + \dots + \beta_{t_0-1} y^{t_0-1} \in \mathbb{F}_q[y]/F(y)$;
- 8 计算 β 在 $\mathbb{F}_q$ 上的极小多项式 M (次数为 t_0) ;
- 9 **if** $\deg M \neq t_0$ **then**
- 10 $\perp$ **return** $\perp$;
- 11 计算 $\text{ord}(\beta)$, 并检查其与 $q^{t_0} - 1$ 的 ℓ -赋值是否相等;
- 12 **if** 不相等 **then**
- 13 $\perp$ **return** $\perp$;
- 14 $g(x) \leftarrow M\left(\left(x - \frac{1}{\eta}\right)^\ell\right) = g_0 + g_1 x + \dots + g_{t_0-1} x^{t_0-1} + x^t$ 在 $\mathbb{F}_q$ 上不可约;
- 15 **return** (η, g) ;

算法1 注释如下。

输入 长度为 $\sigma_1(t_0 + 1)$ 的比特串 $d_0, \dots, d_{\sigma_1(t_0+1)-1}$, $\sigma_1 \geq m$ 。

输出 $\perp$ 或 $\eta \in \mathbb{F}_q^*$ 以及次数为 $t = \ell t_0$ 的首一不可约多项式 $g \in \mathbb{F}_q[x]$ 。

过程

1. 把长度 $\sigma_1(t_0 + 1)$ 的比特串组成 $t_0 + 1$ 段, 对每个 $j \in \{0, \dots, t_0 - 1\}$, 定义

$$\beta_j = \sum_{i=0}^{m-1} d_{\sigma_1 j + i} z^i \in \mathbb{F}_q,$$

$$\eta = \sum_{i=0}^{m-1} d_{\sigma_1 t_0 + i} z^i \in \mathbb{F}_q.$$

其中每组 σ_1 个比特串, 只使用前 m 个比特。如果 $\eta = 0$, 返回 $\perp$ 。

2. 定义

$$\beta = \beta_0 + \beta_1 y + \dots + \beta_{t_0-1} y^{t_0-1} \in \mathbb{F}_q[y]/F(y).$$

3. 计算 β 在 $\mathbb{F}_q$ 上次数为 t_0 的极小多项式 M (设 $\mathbb{F}_q[x]$ 中次数为 t_0 的不可约多项式的个数为 $N_q(t_0)$ (见[28]), 成功概率为 $t_0 N_q(t_0)/q^{t_0}$, 当 $t_0 \geq 16$, 成功概率超过99%)。若 M 的次数不为 t_0 , 返回 $\perp$ 。
4. 计算 $\text{ord}(\beta)$, 使 $\text{ord}(\beta)$ 与 $q^{t_0} - 1$ 的 ℓ -赋值相等; 如果不相等, 返回 $\perp$ 。
5. $M(x^\ell)$ 不可约。令

$$g(x) = M\left(\left(x - \frac{1}{\eta}\right)^\ell\right) = g_0 + g_1 x + \dots + g_{t_0-1} x^{t_0-1} + x^t.$$

注： $M(x)$ 在 $\mathbb{F}_q$ 上不可约，那么 $M(x^\ell)$ 在 $\mathbb{F}_q$ 上不可约当且仅当 $x^\ell - \beta$ 在 $\mathbb{F}_{q^{t_0}}$ 上不可约当且仅当 $\text{ord}(\beta)$ 与 $q^{t_0} - 1$ 的 ℓ -赋值相等(见[28])当且仅当 $\beta \notin (\mathbb{F}_{q^{t_0}})^\ell$ 。那么成功概率为 $1 - 1/\ell$ 。当 $\ell \geq 13$ 时，成功概率超过92.3%。

4.2 域元素生成

Algorithm 2: FieldOrdering

Input: 长度为 $\sigma_2(q-1)$ 的比特串 $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$

Output: $\perp$ 或 $\mathbb{F}_q$ 中 $q-1$ 个不同非零元素序列 $(a_1, \dots, a_{q-1})$

1 把 $\sigma_2(q-1)$ 的比特串分成 $q-1$ 段，每组 σ_2 个比特串构造非负整数：

$$h_i = b_{\sigma_2(i-1)} + 2b_{\sigma_2(i-1)+1} + \dots + 2^{\sigma_2-1}b_{\sigma_2 i-1};$$

2 **if** $h_1, \dots, h_{q-1}$ 不两两不同 **then**

3 **return** $\perp$;

4 按字典序（小前大后）排列 (h_i, i) ，得到 $(h_{\pi(i)}, \pi(i))$;

5 定义

$$a_i = \sum_{j=0}^{m-1} \pi(i)_j z^{m-1-j},$$

其中 $\pi(i) = \sum_{j=0}^{m-1} \pi(i)_j 2^j$;

6 **return** $(a_1, \dots, a_{q-1})$;

算法2 注释如下。

输入 长度为 $\sigma_2(q-1)$ 的比特串 $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ 。

输出 $\perp$ 或 $\mathbb{F}_q$ 中 $q-1$ 个不同非零元素序列 $(a_1, \dots, a_{q-1})$ 。

过程

1. 把 $\sigma_2(q-1)$ 的比特串分成 $(q-1)$ 段，每组 σ_2 个比特串构造非负整数：

$$h_i = b_{\sigma_2(i-1)} + 2b_{\sigma_2(i-1)+1} + \dots + 2^{\sigma_2-1}b_{\sigma_2 i-1},$$

其中每个 $0 \leq h_i < 2^{2m}$ 。

2. 若 $h_1, \dots, h_{q-1}$ 不两两不同，返回 $\perp$ （大概率两两不同）。

3. 按字典序(小前大后)排列 (h_i, i) ，得到 $(h_{\pi(i)}, \pi(i))$ 。

4. 定义

$$a_i = \sum_{j=0}^{m-1} \pi(i)_j z^{m-1-j},$$

其中 $\pi(i) = \sum_{j=0}^{m-1} \pi(i)_j 2^j$ 。

5. 输出 $(a_1, \dots, a_{q-1})$ 。

4.3 定义集排序生成

Algorithm 3: DefinedSet

Input: $\eta \in \mathbb{F}_q^*$ 以及长度为 $\sigma_2(q-1)$ 的比特串 $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$
Output: $\perp$, 或 $\mathbb{F}_q$ 中由 n 个元素组成的集合 $L = \{\alpha_0, \dots, \alpha_{n-1}\}$

- 1 $(a_1, \dots, a_{q-1}) \leftarrow \text{FIELDORDERING}(b_0, \dots, b_{\sigma_2(q-1)-1});$
- 2 **if** *FieldOrdering* 失败 **then**
- 3 **return** $\perp$;
- 4 $A \leftarrow \begin{pmatrix} \zeta & \frac{\zeta+1}{\eta} \\ 0 & 1 \end{pmatrix};$
- 5 $\mathcal{O} \leftarrow [];$
- 6 **for** $i = 1$ **to** $q-1$ **do**
- 7 **if** $a_i + \frac{1}{\eta}$ 不包含在已存储于 $\mathcal{O}$ 的任一轨道中 **then**
- 8 $O_i \leftarrow \left\{ a_i + \frac{1}{\eta}, \zeta a_i + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_i + \frac{1}{\eta} \right\}$
- 9 将 O_i 追加到 $\mathcal{O}$;
- 8 **if** $|\mathcal{O}| = n/\ell$ **then**
- 9 **中断**;
- 10 按 $\mathcal{O}$ 中轨道的存储顺序将其拼接, 形成 L ;
- 11
- 12 $L = \left\{ a_1 + \frac{1}{\eta}, \zeta a_1 + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_1 + \frac{1}{\eta}; \dots; a_{\frac{n}{\ell}} + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_{\frac{n}{\ell}} + \frac{1}{\eta} \right\};$
- 12 **return** L ;

算法3 注释如下。

输入 $\eta \in \mathbb{F}_q^*$ 以及长度为 $\sigma_2(q-1)$ 的比特串 $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ 。

输出 $\perp$ 或 $\mathbb{F}_q$ 中 n 个元构成的集合 $L = \{\alpha_0, \dots, \alpha_{n-1}\}$ 。

过程

1. 由输入比特串调用FieldOrdering, 得到 $(a_1, \dots, a_{q-1})$ 。若失败, 返回 $\perp$ 。
2. 依次选取尚未落入已构造乘法轨道的代表元 $a_i + \frac{1}{\eta}$, 仿射变换 $A = \begin{pmatrix} \zeta & \frac{\zeta+1}{\eta} \\ 0 & 1 \end{pmatrix}$, 并构造仿射平移轨道

$$O_i = \left\{ a_i + \frac{1}{\eta}, \zeta a_i + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_i + \frac{1}{\eta} \right\}.$$

3. 取前 $\frac{n}{\ell}$ 个这样的轨道, 按轨道顺序输出

$$L = \left\{ a_1 + \frac{1}{\eta}, \zeta a_1 + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_1 + \frac{1}{\eta}; \dots; a_{\frac{n}{\ell}} + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_{\frac{n}{\ell}} + \frac{1}{\eta} \right\}.$$

4.4 矩阵生成

Algorithm 4: MatGen

Input: $\Gamma = (g, L, \eta)$, 其中 $g \in \mathbb{F}_q[x]$ 是次数为 $t = \ell t_0$ 的首一不可约多项式, $L = \{\alpha_0, \dots, \alpha_{n-1}\}$ 和 $\eta \in \mathbb{F}_q^*$

Output: $\perp$ 或 $(\psi(T_1), T_2)$ 和 $\Gamma' = (g, L', \eta)$

1 计算 $\mathbb{F}_q$ 上的 $t \times n$ 矩阵 $H_T = (h_{i,j})$ 并做初等行变换:

$$H_T = \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0^{t-1} + \eta \alpha_0^t} & \frac{g(\alpha_1)}{\alpha_1^{t-1} + \eta \alpha_1^t} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}^{t-1} + \eta \alpha_{n-1}^t} \\ \frac{\alpha_0^{t-1} + \eta \alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta \alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta \alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix} \sim \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{1} & \frac{g(\alpha_1)}{1} & \cdots & \frac{g(\alpha_{n-1})}{1} \end{pmatrix};$$

2 将 H_T 的前 $t-1$ 行中每个元素 $h_{i,j}$ 表示为

$$u_0 + u_1 z + \cdots + u_{m-1} z^{m-1}.$$

这样, $h_{i,j}$ 替换为二元列向量 $(u_0, u_1, \dots, u_{m-1})$; 最后一行全1 不展开, 得到

$$\hat{H}_T \in \mathbb{F}_2^{(m(t-1)+1) \times n}.$$

3 $u \leftarrow m(t-1) + 1$;

4 利用 $\hat{H}_T$ 构造 ℓ -分块循环矩阵 $H_1 \in \mathbb{F}_2^{\ell u \times n}$: 对 $\hat{H}_T$ 的每一行作分块循环移位;

5 对 H_1 做 ℓ -循环块高斯消元;

6 **if** 成功 **then**

7 得到列的块置换 τ 和可逆矩阵 Q_0 , 使得

$$Q_0 \tau(H_1) = \begin{pmatrix} H \\ 0 \end{pmatrix}$$

其中

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}, \quad r_0 \equiv u \pmod{\ell};$$

8 从 T_1 中提取第 $1, \ell+1, 2\ell+1, \dots, (u-r_0)-\ell+1$ 行, 构成

$$\psi(T_1) \in \mathbb{F}_2^{\frac{u-r_0}{\ell} \times (n-u+r_0)};$$

9 $L' \leftarrow \tau(L)$;

// 存在 $u \times u$ 可逆矩阵 Q 和同一列置换 τ , 使得 $Q\tau(\hat{H}_T) = H$

10 **return** $(\psi(T_1), T_2)$, $\Gamma' = (g, L', \eta)$;

11 **else**

12 **return** $\perp$;

算法4 注释如下。

输入 $\Gamma = (g, L, \eta)$, 其中 $g \in \mathbb{F}_q[x]$ 是次数为 $t = \ell t_0$ 的首一不可约多项式, $L = \{\alpha_0, \dots, \alpha_{n-1}\}$ 和 $\eta \in \mathbb{F}_q^*$ 。

输出 $\perp$ 或 $(\psi(T_1), T_2)$ 和 $\Gamma' = (g, L', \eta)$ 。

过程

1. 计算 $\mathbb{F}_q$ 上的 $t \times n$ 矩阵

$$H_T = (h_{i,j}) = \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{\alpha_0^{t-1} + \eta \alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta \alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta \alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix}$$

进一步, 对 H_T 做行初等变换, 可得:

$$H_T \sim \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ 1 & 1 & \cdots & 1 \end{pmatrix}$$

注意到最后一行是全1行。

2. 将 H_T 的前 $t-1$ 行中每个元素 $h_{i,j}$ 表示为

$$u_0 + u_1 z + \cdots + u_{m-1} z^{m-1}$$

这样 $h_{i,j}$ 替换为二元列向量 $(u_0, u_1, \dots, u_{m-1})$ 。最后一行全1 不展开。得到

$$\hat{H}_T \in \mathbb{F}_2^{(m(t-1)+1) \times n},$$

其中大概率情况下矩阵 $\hat{H}_T$ 的秩等于 $m(t-1)+1$ 。

3. 令 $u = m(t-1)+1$ 。利用 $\hat{H}_T$ 构造 ℓ -分块循环矩阵 $H_1 \in \mathbb{F}_2^{\ell u \times n}$: 对 $\hat{H}_T$ 的每一行作分块循环移位。
4. 对 H_1 做 ℓ -循环块高斯消元。若成功, 得到列的块置换 τ 和可逆矩阵 Q_0 , 使得

$$Q_0 \tau(H_1) = \begin{pmatrix} H \\ 0 \end{pmatrix},$$

其中

$$\hat{H}_T \sim H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}, \quad r_0 \equiv u \pmod{\ell},$$

这里 T_1 为 $(u-r_0) \times (n-u+r_0)$ 阶 ℓ -分块循环矩阵, T_2 为 $r_0 \times (n-u+r_0)$ 阶行满秩矩阵(目前尝试5000次均成功, 鉴于BIG QUAKE方案的观点, 这是大概率成功的事件)。从 T_1 中提取第 $1, \ell+1, 2\ell+1, \dots, (u-r_0)-\ell+1$ 行, 构成

$$\psi(T_1) \in \mathbb{F}_2^{\frac{u-r_0}{\ell} \times (n-u+r_0)}.$$

5. 输出:

$$(\psi(T_1), T_2), \Gamma' = (g, L', \eta),$$

这里 $L' \leftarrow \tau(L)$, 存在 $u \times u$ 可逆矩阵 Q 和同一列置换 τ , 使得

$$Q\tau(\hat{H}_T) = H.$$

若失败, 返回 $\perp$ 。

注: MatGen 输出的公开矩阵是混合系统型矩阵。只有 T_1 按 ℓ -分块循环矩阵压缩; T_2 的 r_0 行按普通二元矩阵保存。不需要保存系统化过程中产生的辅助矩阵 Q_0, Q, τ 。

4.5 固定重量向量生成

Algorithm 5: FixedWeight

Input: 参数集和内部随机性

Output: 权重为 $w = (t-1)/2$ 的向量 $e \in \mathbb{F}_2^n$

```

1 选择  $\rho$ : 若  $n = q$ , 则令  $\rho = w$ ; 若  $q/2 \leq n < q$ , 则令  $\rho = 2w$ ; 依此类推;
2 repeat
3   生成  $\sigma_1 \rho$  个均匀随机比特  $b_0, \dots, b_{\sigma_1 \rho - 1}$ ;
4   for  $j = 0$  to  $\rho - 1$  do
5      $d_j \leftarrow \sum_{i=0}^{m-1} b_{\sigma_1 j + i} 2^i$ ;
6   令  $D$  为  $d_0, \dots, d_{\rho-1}$  中属于  $\{0, \dots, n-1\}$  的两两不同元素所组成的有序列表;
7 until  $|D| \geq w$ ;
8 初始化  $e \leftarrow (0, \dots, 0) \in \mathbb{F}_2^n$ ;
9 for  $D$  中的前  $w$  个位置  $j$  do
10   $e_j \leftarrow 1$ ;
11 return  $e$ ;
```

4.6 编码

Algorithm 6: Encode

Input: 重量为 $w = (t-1)/2$ 的 $e \in \mathbb{F}_2^n$, 公钥 $(\psi(T_1), T_2)$

Output: 密文校验子 $C \in \mathbb{F}_2^u$

1 从 $\psi(T_1)$ 恢复 ℓ -分块循环矩阵 T_1 , 并定义

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix};$$

2 return

$$C = He^T \in \mathbb{F}_2^u;$$

4.7 译码

Algorithm 7: Decode

Input: 密文 $C \in \mathbb{F}_2^m$, 私钥 $\Gamma' = (g, L', \eta)$, 以及公钥 $(\psi(T_1), T_2)$
Output: 重量为 $w = (t-1)/2$ 的向量 $e \in \mathbb{F}_2^n$, 或 $\perp$

- 1 使用 Γ' 重新执行 MatGen 的步骤1 和步骤2, 以得到 $\hat{H}_T$;
- 2 使用 $(\psi(T_1), T_2)$ 执行 Encode 的步骤1, 以恢复 H ;
- 3 计算满足 $\hat{H}_T = Q^{-1}H$ 的 Q^{-1} ;
- 4 计算 $\hat{H}_T e^T = Q^{-1}H e^T = Q^{-1}C$, 其中使用 $C = H e^T$;
- 5 使用基 $1, z, \dots, z^{m-1}$ 将结果恢复为 $\mathbb{F}_q$ 中的元素;
- 6 按顺序恢复后续坐标, 并将对应于最后一行的坐标保持为 $\mathbb{F}_2$ 中的单位坐标;
- 7 设

$$S = H_T e^T;$$
- 8 使用 [36, 37] 中的解码算法, 令 $e \leftarrow \text{TWISTEDGOPPADECODE}(S, g, L', \eta)$;
- 9 **if** e 不存在或 $\text{wt}(e) \neq w$ **then**
- 10 **return** $\perp$;
- 11 **return** e ;

注: 经过多次测试, 该方案采用的扭 Goppa 码的最小距离均为 $t+1$, 此译码算法是最优的, 不同于人们通常采用的 Goppa 码的 Patterson 算法 [33]。

5 密钥封装机制

5.1 密钥生成

Algorithm 8: KeyGen

Input: 参数集和内部随机性
Output: 公钥 pk 和私钥 sk

- 1 均匀随机采样一个 λ 比特种子 δ ;
- 2 **return** $\text{SeededKeyGen}(\delta)$;

这里, 算法 $\text{SeededKeyGen}(\delta)$ 定义如下。

Algorithm 9: SeededKeyGen

Input: 种子 δ
Output: 公钥 pk 和私钥 sk

- 1 计算 $E = G(\delta)$, 得到长度为

$$n + \sigma_2(q-1) + \sigma_1(t_0+1) + \lambda$$
 的比特串;
- 2 令 δ' 为 E 的最后 λ 位, 令 s 为 E 的前 n 位;
- 3 令 ρ 为 E 中接下来的 $\sigma_1(t_0+1)$ 位;
- 4 $(\eta, g) \leftarrow \text{GoppaPolynomial}(\rho)$;
- 5 **if** $(\eta, g) = \perp$ **then**
- 6 $\delta \leftarrow \delta'$;
- 7 重新开始;
- 8 令 ρ_L 为 E 中接下来的 $\sigma_2(q-1)$ 位;
- 9 $L \leftarrow \text{DefinedSet}(\rho_L)$;
- 10 **if** $L = \perp$ **then**
- 11 $\delta \leftarrow \delta'$;
- 12 重新开始;

Algorithm 9: SeededKeyGen (continued)

```
13 令  $\Gamma = (g, L)$ ;  
14  $(\psi(T_1), T_2, \Gamma') \leftarrow \text{MatGen}(\Gamma)$ ;  
15 if  $(\psi(T_1), T_2, \Gamma') = \perp$  then  
16    $\delta \leftarrow \delta'$ ;  
17   重新开始;  
18 输出公钥pk 和私钥sk 如下:
```

$$\text{pk} = (\psi(T_1), T_2), \quad \text{sk} = (\delta, \Gamma', s).$$

5.2 封装

Algorithm 10: Encaps

```
Input: 公钥  $\text{pk} = (\psi(T_1), T_2)$   
Output: 密文  $C$  和会话密钥  $K$   
1  $e \leftarrow \text{FIXEDWEIGHT}$ , 其中  $e \in \mathbb{F}_2^n$  且  $\text{wt}(e) = w = (t-1)/2$ ;  
2  $C \leftarrow \text{ENCODE}(e, \psi(T_1), T_2)$ ;  
3  $K \leftarrow \mathbf{H}(1, e, C)$ ;  
4 return  $(C, K)$ ;
```

5.3 解封装

Algorithm 11: Decaps

```
Input: 密文  $C$  和私钥  $\text{sk} = (\delta, \Gamma', s)$   
Output: 会话密钥  $K$   
1  $b \leftarrow 1$ ;  
2  $e \leftarrow \text{DECODE}(C, \Gamma')$ ;  
3 if  $e = \perp$  then  
4    $e \leftarrow s$ ;  
5    $b \leftarrow 0$ ;  
6  $K \leftarrow \mathbf{H}(b, e, C)$ ;  
7 return  $K$ ;
```

6 安全性考虑

QCTM 方案的设计同时借鉴了Classic McEliece方案[11]和BIG QUAKE方案[1]。与Classic McEliece方案相比, 区别在于对公钥矩阵的一部分使用准群环结构进行了压缩, 因此总体公钥尺寸大幅度减少。与BIG QUAKE方案相比, 区别在于没有对整个公钥矩阵使用准循环结构, 而是在Goppa校验矩阵的最后一行引入了扭项, 因此总体公钥尺寸只有少量增加。这样做的期望是取得安全性和效率(特别是公钥尺寸)方面的平衡: 扭项的引入和对整体准循环结构的破坏, 旨在规避可能存在的针对准循环结构的不变子码攻击, 局部准循环结构的保持又为大幅压缩公钥尺寸提供可能。总体上, QCTM方案的安全性介于Classic McEliece方案和BIG QUAKE方案之间。

下面我们分别从底层困难问题、实际安全性测算依据、参数选择三个方面进行分析。

6.1 底层困难问题

QCTM 的安全性依赖如下问题在所列参数下的困难性:

- 给定公开二元校验矩阵和校验子, 寻找重量为 w 的错误向量的校验子译码问题;
- 从公开混合系统型矩阵 $(\psi(T_1), T_2)$ 恢复私钥扭Goppa 结构 Γ' 的困难性;
- 区分由上述KeyGen 生成的公开矩阵与相同尺寸随机混合系统型矩阵的困难性;
- 所用杂凑函数、XOF 或伪随机生成器的随机预言机或伪随机性假设。

本文方案的安全性建立在编码理论中的若干基本困难问题之上。由于本方案采用具有隐藏代数结构的扭Goppa 码，并进一步引入准循环结构，因此其安全性不仅依赖于通用校验子译码问题的困难性，还依赖于私钥码结构隐藏性、准循环校验子译码问题以及公开密钥不可区分性等假设[3]。

定义1: 校验子译码问题 $\text{SD}(n, k, w)$. 给定正整数 n, k, w , 其中 $0 < k < n$ 。设

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

为一个公开的二元校验矩阵, 设

$$s \in \mathbb{F}_2^{n-k}$$

为一个公开校验子。校验子译码问题 $\text{SD}(n, k, w)$ 要求寻找一个错误向量

$$e \in \mathbb{F}_2^n$$

使得

$$\text{wt}(e) = w, \quad He^T = s,$$

其中, $\text{wt}(e)$ 表示向量 e 的汉明重量。

该问题刻画了攻击者在只知道公开校验矩阵和校验子的情况下, 恢复低重量错误向量的困难性。在Niederreiter 型或McEliece 型编码密码中, 密文或封装值通常可以表示为某个低重量错误向量对应的校验子, 因此若攻击者能够有效求解 $\text{SD}(n, k, w)$, 就可以直接恢复封装错误向量, 进而破坏方案的机密性。现有针对该问题的主要攻击方法是信息集译码攻击, 包括Prange、Stern、Dumer、Ball-Collision、BJMM 和May-Ozerov 等算法。因此, 参数组 (n, k, w) 的经典安全强度通常由这些攻击算法中的最低复杂度决定。

定义2: 扭Goppa 私钥结构恢复问题. 设 Γ' 表示密钥生成算法中使用的扭Goppa 私钥结构。该结构包含支撑集、Goppa 多项式、扭曲参数、置换变换以及与高效译码相关的秘密代数信息。密钥生成算法根据 Γ' 构造具有高效译码算法的私钥码, 并经过隐藏变换、系统化处理和二元展开映射等步骤, 生成公开混合系统型矩阵

$$H_{\text{pub}} = (\psi(T_1), T_2).$$

扭Goppa 私钥结构恢复问题是指: 给定公开矩阵

$$H_{\text{pub}} = (\psi(T_1), T_2),$$

恢复原始私钥结构 Γ' , 或者恢复一个等价私钥结构 Γ'' , 使得 Γ'' 所定义的码与公开矩阵对应的码等价, 并且能够支持有效译码。

该问题对应编码密码中的结构恢复攻击或密钥恢复攻击。与校验子译码问题不同, 结构恢复攻击的目标不是求解某一次封装中的错误向量, 而是从公开矩阵中恢复隐藏的私钥码结构。一旦攻击者获得 Γ' 或某个等价结构, 就可以像合法解封装者一样对任意校验子进行高效译码。因此, 私有扭Goppa 结构恢复问题的困难性保证了公开矩阵不会泄露足以恢复私钥译码能力的代数信息。

定义3: 搜索型 ℓ -准循环校验子译码问题 $\ell\text{-QCSD}(n, k, w)$. 给定正整数 n, k, w, ℓ , 其中 $0 < k < n$ 。设

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

为一个系统型 ℓ -准循环码的公开二元校验矩阵, 设

$$s \in \mathbb{F}_2^{n-k}$$

为一个公开校验子。搜索型 ℓ -准循环校验子译码问题, 记为

$$\ell\text{-QCSD}(n, k, w),$$

要求寻找一个错误向量

$$e \in \mathbb{F}_2^n$$

使得

$$\text{wt}(e) = w, \quad He^T = s.$$

与普通校验子译码问题相比, ℓ -准循环校验子译码问题中的公开校验矩阵额外具有准循环结构。准循环结构能够显著压缩公钥规模, 但同时也可能被攻击者利用, 从而带来一定的复杂度降低。例如, DOOM 类攻击可以利用准循环结构产生一个小幅加速因子。因此, 在安全性估计中, 通常先按照通用信息集译码攻击评估 (n, k, w) 的基础复杂度, 再根据准循环阶数 ℓ 扣除相应的结构折扣。已有研究通常认为, 对于合理选择的参数, 准循环结构只会带来有限影响, 并不会根本改变校验子译码问题的困难性。

定义4: 准循环Goppa 码公开密钥不可区分问题DIQCG. 设 $\mathcal{D}_{\text{QCG}}$ 表示由密钥生成算法输出的 ℓ -准循环Goppa 码公开矩阵分布, 即

$$H_{\text{QCG}} \leftarrow \mathcal{D}_{\text{QCG}}.$$

设 $\mathcal{D}_{\text{rand}}$ 表示与其具有相同尺寸、相同秩条件和相同系统型 ℓ -准循环形式的随机码矩阵分布, 即

$$H_{\text{rand}} \leftarrow \mathcal{D}_{\text{rand}}.$$

准循环Goppa 码公开密钥不可区分问题要求攻击者在给定矩阵 M 时, 判断 M 是来自真实的准循环Goppa 码公开密钥分布 $\mathcal{D}_{\text{QCG}}$, 还是来自随机准循环码矩阵分布 $\mathcal{D}_{\text{rand}}$ 。形式化地, 攻击者 $\mathcal{A}$ 的区分优势定义为

$$\text{Adv}_{\mathcal{A}}^{\text{DIQCG}} = \left| \Pr_{H_{\text{QCG}} \leftarrow \mathcal{D}_{\text{QCG}}} [\mathcal{A}(H_{\text{QCG}}) = 1] - \Pr_{H_{\text{rand}} \leftarrow \mathcal{D}_{\text{rand}}} [\mathcal{A}(H_{\text{rand}}) = 1] \right|.$$

若对任意概率多项式时间攻击者 $\mathcal{A}$, 上述优势均为可忽略量, 则称由密钥生成算法得到的 ℓ -准循环Goppa 码公开矩阵与相同形式的随机准循环码矩阵计算不可区分。

该问题刻画的是公开密钥是否泄露其背后的Goppa 码代数结构。如果攻击者能够区分真实公开矩阵和随机准循环矩阵, 就说明公开密钥中仍然存在可被识别的结构特征, 这可能进一步发展为结构恢复攻击或密钥恢复攻击。因此, 公开密钥不可区分性是保证准循环Goppa 型编码密码结构安全性的关键假设。

综上, 本文方案的安全性可以从两个层面理解: 一方面, 攻击者需要面对低重量错误向量恢复问题, 即 $\text{SD}(n, k, w)$ 或具有准循环结构的 ℓ -QCSD (n, k, w) ; 另一方面, 攻击者还可能尝试从公开矩阵中识别或恢复隐藏的扭Goppa 结构, 因此需要依赖私有结构恢复困难性和公开密钥不可区分假设。

6.2 实际安全性测算依据

本提案采用近年来广泛使用的校验子评估器 `SDEstimator` [16] 来进行参数集选择及方案实际安全性评估。¹ 该评估器涵盖了主流的信息集译码 (Information Set Decoding, ISD) 攻击算法, 包括 Prange、Stern、Dumer、Ball-Collision、BJMM、BJMM-pdw、May-Ozerov 和 Both-May 等。这些攻击算法的基本目标是在给定校验矩阵 H 和校验子 $s = He^T$ 的情况下, 恢复一个重量为 w 的错误向量 e 。对于给定参数组, 我们运行 `SDEstimator`, 并取所有攻击算法中的最低时间复杂度作为对应参数下的经典安全强度。

- Prange 算法[34]是最早也是最基本的信息集译码算法。其核心思想是随机选取一个信息集, 并假设错误向量在该信息集上的位置为空, 从而通过线性代数运算恢复候选错误向量。若所选信息集恰好避开所有错误位置, 则译码成功。因此, Prange 算法的复杂度主要由随机选中无错信息集的概率决定。虽然该算法结构简单, 但它为后续所有改进型ISD 攻击提供了基准复杂度。
- Stern 算法[35]在Prange 算法的基础上引入了碰撞搜索思想。该算法允许信息集中存在少量错误, 并将搜索空间划分为若干部分, 通过构造和匹配中间列表来寻找满足综合征条件的低重量错误向量。与Prange 算法相比, Stern 算法利用时间-空间折中显著降低了攻击复杂度, 但需要额外的存储空间来保存和匹配中间列表。
- Dumer 算法[15]进一步改进了Stern 算法中的分块和碰撞策略。其主要思想是通过更精细的线性变换和分块处理, 将原始译码问题化为较小规模的子问题, 再利用列表碰撞方法寻找候选错误向量。Dumer 算法通常在时间复杂度上优于Stern 算法, 是早期高效ISD 攻击中的重要改进方案。

¹最新实现代码见: https://github.com/Crypto-TII/syndrome_decoding_estimator

- Ball-Collision 算法[10]是一类基于球碰撞思想的ISD 攻击。与传统Stern 或Dumer 方法中固定重量分解的方式不同, Ball-Collision 算法通过在一定半径的汉明球内搜索候选向量, 并利用碰撞匹配来提高低重量错误恢复的效率。该方法可以看作是对列表碰撞策略的进一步优化, 在某些参数区域能够获得比Stern 和Dumer 更低的攻击复杂度。
- BJMM 算法, 即Becker-Joux-May-Meurer 算法[4], 是现代信息集译码攻击中的重要代表。该算法引入了表示技术 (representation technique), 即允许同一个目标错误向量通过多种方式表示为若干较低重量向量的组合, 从而增加碰撞成功概率并降低整体搜索复杂度。BJMM 算法通常能够显著优于Prange、Stern 和Dumer 等传统ISD 算法, 因此在编码密码参数安全性评估中具有重要地位。
- BJMM-pdw 和BJMM-dw 是BJMM 算法的进一步变体[4], 并作为BJMM 族算法在SDEstimator[16] 中用于参数安全性估计。其中, pdw 通常表示partial distance weight, 即通过引入部分距离或部分重量约束来优化列表构造和碰撞搜索过程; dw 则与距离权重分布相关。这些变体的目的在于更精细地利用错误向量的重量分布特征, 从而在特定参数范围内进一步降低攻击复杂度。因此, 在安全性评估中需要将BJMM 及其变体同时纳入比较。
- May-Ozerov 算法[29]是在BJMM 思想基础上结合最近邻搜索技术得到的进一步改进型ISD 攻击。该算法将部分碰撞搜索问题转化为最近邻搜索问题, 并利用更高效的数据结构和搜索策略降低时间复杂度。在许多编码密码参数设置下, May-Ozerov 算法往往给出所有经典ISD 攻击中最低的复杂度, 因此它经常成为决定参数经典安全等级的最强攻击算法。
- Both-May 算法[12]可以看作是May 系列信息集译码攻击的进一步变体, 其主要目标是在时间复杂度与空间复杂度之间取得更优折中。该算法结合了多种列表构造、碰撞搜索和最近邻搜索思想, 在某些参数区域能够接近或优于May-Ozerov 攻击的复杂度。因此, 将Both-May 攻击纳入评估可以使安全性估算更加全面。

综上, 本文在参数安全性评估中并不依赖单一攻击算法, 而是对上述多种经典ISD 攻击进行统一估算, 并取其中最低的攻击复杂度作为参数组的经典安全强度。设 $T_A(n, k, w)$ 表示攻击算法 A 在参数组 (n, k, w) 下的时间复杂度, 则经典安全强度定义为

$$\lambda_{\text{classical}} = \min_A \log_2 T_A(n, k, w),$$

其中 A 遍历Prange、Stern、Dumer、Ball-Collision、BJMM、BJMM-pdw、May-Ozerov 和Both-May 等信息集译码攻击算法。若进一步考虑准循环结构可能带来的DOOM 加速, 则在经典安全强度的基础上扣除

$$\frac{1}{2} \log_2 \ell$$

的结构折扣, 从而得到最终安全等级。在QCTM方案中, 我们固定 $\ell = 19$, 因此 $\frac{1}{2} \log_2(19) \approx 2.12$ 。

三组提交参数集的时间安全性测算结果如下。

• 第1 组参数实例QCTM128

对于第1 组参数实例, 带入码长 $n = 10070$ 、维数 $k = 4957$ 、准循环阶数 $l = 19$ 以及错误重量 $w = 142$, 得到的安全性评估如表 1 所示。

Table 1: 第一组参数安全性评估结果

攻击算法	基础安全强度	最终安全强度
BallCollision	155.62 bits	153.50 bits
BJMMdw	153.73 bits	151.61 bits
BJMMpdw	153.71 bits	151.59 bits
BJMM	154.02 bits	151.89 bits
BJMMplus	153.68 bits	151.56 bits
BothMay	153.01 bits	150.88 bits
Dumer	155.60 bits	153.48 bits
MayOzerov	152.99 bits	150.87 bits
Prange	176.14 bits	174.01 bits
Stern	154.64 bits	152.52 bits

在所考虑的攻击算法中，**MayOzerov** 给出最低攻击复杂度。因此，该参数组的最终安全强度为**150.87 bits**，高于**128-bit** 安全级别。

- **第2 组参数实例QCTM256**

对于第2 组参数实例，带入码长 $n = 19000$ 、维数 $k = 9783$ 、准循环阶数 $l = 19$ 以及错误重量 $w = 256$ ，得到的安全性评估如表 2 所示。

Table 2: 第二组参数安全性评估结果

攻击算法	基础安全强度	最终安全强度
BallCollision	283.77 bits	281.64 bits
BJMMdw	277.10 bits	274.98 bits
BJMMpdw	277.11 bits	274.98 bits
BJMM	276.50 bits	274.38 bits
BJMMplus	276.94 bits	274.81 bits
BothMay	275.95 bits	273.83 bits
Dumer	283.28 bits	281.16 bits
MayOzerov	275.03 bits	272.90 bits
Prange	308.28 bits	306.15 bits
Stern	282.76 bits	280.64 bits

在所考虑的攻击算法中，**MayOzerov** 给出最低攻击复杂度。因此，该参数组的最终安全强度为**272.90 bits**，高于**256-bit** 安全级别。

- **第3 组参数实例QCTM512**

对于第3 组参数实例，带入码长 $n = 38000$ 、维数 $k = 19891$ 、准循环阶数 $l = 19$ 以及错误重量 $w = 503$ ，得到的安全性评估如表 3 所示。

Table 3: 第三组参数安全性评估结果

攻击算法	基础安全强度	最终安全强度
BallCollision	553.96 bits	551.84 bits
BJMMdw	535.02 bits	532.89 bits
BJMMpdw	534.75 bits	532.63 bits
BJMM	532.42 bits	530.30 bits
BJMMplus	534.58 bits	532.46 bits
BothMay	533.88 bits	531.76 bits
Dumer	553.90 bits	551.77 bits
MayOzerov	532.95 bits	530.83 bits
Prange	584.47 bits	582.34 bits
Stern	553.70 bits	551.57 bits

在该大规模参数下，**BJMM** 给出最低攻击复杂度。因此，该参数组的最终安全强度为**530.30 bits**，高于**512-bit** 安全级别。

6.3 结构攻击

根据报告[2]，BIG QUAKE 未被选入NIST-PQC 第二轮征集的主要原因之一是由于文献[19] 中提出的结构折叠攻击，BIG QUAKE 可能存在安全隐患。为克服这一问题，我们旨在通过在校验矩阵中引入扭曲项，来规避此类结构折叠攻击。具体细节如下。

QCTM方案 不能直接采用文献[19]的折叠手段攻击。设QCTM的方案扭准循环Goppa码 $\Gamma(g, L, \eta)$ 的校验矩阵为 $\mathbb{F}_q$ 上的 $t \times n$ 矩阵:

$$H_T = \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0^{t-1} + \eta\alpha_0^t} & \frac{g(\alpha_1)}{\alpha_1^{t-1} + \eta\alpha_1^t} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}^{t-1} + \eta\alpha_{n-1}^t} \\ \frac{\alpha_0^{t-1} + \eta\alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta\alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta\alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix}.$$

它也是准循环扭GRS码 $\mathcal{C} = GRS_{n-t}(g, L, \eta)$ 的校验矩阵。由文献[22]知道, 当 $a = \sum_{j=0}^{n-1} \alpha_j \neq 0$ 时, 此码的生成矩阵 $G_{n-t}(g, L, \eta)$ 为:

$$\begin{pmatrix} \frac{g(\alpha_0)}{v_0} & \frac{g(\alpha_1)}{v_1} & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \\ \frac{g(\alpha_0)}{v_0} \alpha_0 & \frac{g(\alpha_1)}{v_1} \alpha_1 & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \alpha_{n-1} \\ \vdots & \vdots & & \vdots \\ \frac{g(\alpha_0)}{v_0} \alpha_0^{n-t-2} & \frac{g(\alpha_1)}{v_1} \alpha_1^{n-t-2} & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \alpha_{n-1}^{n-t-2} \\ \frac{g(\alpha_0)}{v_0} \left(\alpha_0^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_0^{n-t} \right) & \frac{g(\alpha_1)}{v_1} \left(\alpha_1^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_1^{n-t} \right) & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \left(\alpha_{n-1}^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_{n-1}^{n-t} \right) \end{pmatrix},$$

这里 $m(x) = \sum_{j=0}^{n-1} (x - \alpha_j)$ 和每个 $v_i = m'(\alpha_i) = \sum_{j=0, j \neq i}^{n-1} (x - \alpha_j)$. 当 $a = \sum_{j=0}^{n-1} \alpha_j = 0$ 时, $\mathcal{C} = GRS_{n-t}(g, L, \eta)$ 也是扭GRS码。

由于构造特殊性, $GRS_{n-t}(g, L, \eta)$ 是准循环的扭GRS码, 此码可以折叠, 但是他的折叠码既不是GRS也不是扭GRS码, 因为它不满足[19][定理1], 无法直接采用折叠码方式攻击。

具体如下:

$$\Gamma(g, L, \eta) = GRS_{n-t}(g, L, \eta) \cap \mathbb{F}_2^n.$$

已知 $\Gamma(g, L, \eta)$ 有自同构 σ , 所以由折叠码性质,

$$\overline{\Gamma(g, L, \eta)}^\sigma \subset \overline{GRS_{n-t}(g, L, \eta)}^\sigma \cap \mathbb{F}_2^n.$$

事实上, 这个折叠码 $\overline{GRS_{n-t}(g, L, \eta)}^\sigma$ 不是GRS码也不是扭GRS码, 因此该码不能直接采用折叠方式进行攻击。

6.4 解封装失败分析

若Decode 成功, 则Encaps 和Decaps 使用同一 e 与同一 C , 得到相同的

$$K = \mathbf{H}(1, e, C).$$

若Decode 失败, 则Decaps 使用私钥中的 s 输出

$$K = \mathbf{H}(0, s, C).$$

该分支使解封装接口对任意密文均有定义, 并与提交实现的KEM 行为一致。因此, QCTM 方案没有解封装失败。

7 参考和优化实现

7.1 不可约多项式 $f(z)$ 和 $F(y)$ 的实例化

考虑到QCTM需要的准循环扭Goppa码结构设计需要, 涉及的两个不可约多项式

$$f(z) \in \mathbb{F}_2[z] \text{ 和 } F(y) \in \mathbb{F}_q[y]$$

的选择要与具体的参数实例化相适应, 其中 $\mathbb{F}_q \triangleq \mathbb{F}_2[z]/(f(z))$ 。同时考虑到实现的效率, 我们为三组参数选择如下不可约多项式。

参数实例	m	n	ℓ	t_0	t	$f(z)$	$F(y)$
QCTM128	18	10070	19	15	285	$z^{18} + z^3 + 1$	$y^{15} + y^7 + z^7$
QCTM256	18	19000	19	27	513	$z^{18} + z^7 + 1$	$y^{27} + z$
QCTM512	18	38000	19	53	1007	$z^{18} + z^3 + 1$	$y^{53} + y^6 + z$

7.2 矩阵验证

实现中的MatGen 对以下条件进行验证:

- $\hat{H}_T$ 的行数为 $u = m(t-1) + 1$;
- $r_0 = u \bmod \ell$, 并且 T_2 含 r_0 行;
- T_1 的右侧宽度为 $n - (u - r_0) = n - u + r_0$, 不是 $n - u$;
- T_1 的公开部分满足 ℓ -分块循环一致性;

7.3 参考实现

提交包提供C 语言参考实现, 目录为

Implementations/Reference_Implementation.

实现提供crypto_kem_keypair、crypto_kem_enc、crypto_kem_dec 接口, 并提供商用密码算法提交API 适配文件KEM_AlgorithmInstance.h 和KEM_AlgorithmInstance.c。

7.4 优化实现

提交包提供C 语言优化实现, 目录为

Implementations/Optimized_Implementation.

优化后的实现保持了与参考实现相同的crypto_kem_keypair、crypto_kem_enc 和crypto_kem_dec 接口, 但在主要热点路径上减少了重复的计算工作。具体而言, 封装使用缓存的公钥列表示形式, 而解封装则重用已解码的密钥/Goppa 结构, 避免每次调用时都重新构建。优化后的实现包含三个参数集目录: QCTM128、QCTM256 和QCTM512。

8 性能与尺寸

8.1 算法性能 (千时钟周期数)

测试在如下环境下完成: 测试机为MSI GP66 Leopard 11UG, 处理器为Intel Core i7-11800H @ 2.30GHz, 8 个物理核心、16 个逻辑线程, 物理内存为32 GiB。宿主系统为64 位Microsoft Windows 11 家庭中文版 (版本10.0.22631); 测试运行于WSL2 中的Ubuntu 24.04 LTS, Linux 内核版本为6.18.33.2-microsoft-standard-WSL2, 目标架构为amd64。性能测试使用SUPERCOP 20220506, 编译器为GCC 13.3.0, 主要编译选项为-march=native -mtune=native -O3 -fomit-frame-pointer -fwrapv -fPIC -fPIE -fcommon。SHAKE256 采用SUPERCOP 构建目录中的libkeccak.a 实现, 并通过libkeccak.a.headers/SimpleFIPS202.h 提供的接口调用; 本次编译产物中FIPS202_SHAKE256 由该接口封装并调用SHAKE256。该选择与此前复现Classic McEliece 时使用的SHAKE256 实现保持一致。SUPERCOP 记录的计时实现为amd64rdpmc, 处理器频率为2,303,999,000 cycles/s。

参数集	密钥生成	封装	解封装
QCTM128	11,795,687.2	183.4	3,063.1
QCTM256	62,776,212.1	425.2	10,593.5
QCTM512	880,022,864.4	3,326.9	54,024.2

表中数值单位为千时钟周期数, 取自SUPERCOP 输出的avg_kcycles。每个参数集的密钥生成、封装和解封装均包含100 个原始时钟周期样本。

8.2 参数尺寸（字节数）

实例	公钥字节	私钥字节	密文字节	共享密钥字节
QCTM128	167,987	192,545	640	32
QCTM256	595,662	641,942	1,153	32
QCTM512	2,374,727	2,467,243	2,264	64

8.3 已知答案测试

提交包提供三个实例的KAT 文件:

- 第1 组参数实例QCTM128 的KAT 文件
- 第2 组参数实例QCTM256 的KAT 文件
- 第3 组参数实例QCTM512 的KAT 文件

每个KAT 文件均包含公钥、私钥、密文和共享密钥样例，并可由对应实现重新生成。

9 创新性声明

该方案的主要创新如下:

1. 引入扭项抵御已知结构折叠攻击。其底层码族为准循环扭Goppa码[36]，其优势之一在于：引入扭项打破了由[19]提出的发起已知结构折叠攻击的条件。

2. 引入局部准循环性减少公钥尺寸。公钥矩阵是一种混合式系统矩阵，由局部准循环的主部分和普通的尾部行组成。该方案无需使用 $\hat{H}_t$ 的全部 $m(t-1)+1$ 行来支持准循环压缩。系统化缩减仅需前 $u-r_0$ 行构成 ℓ -块循环主部分 T_1 ，其余 r_0 行则作为普通尾部行 T_2 存储。这种设计避免了将普通二进制秩与块兼容秩混淆，同时也防止错误地声称整个块矩阵可以纯以准循环方式压缩。

3. 提出扭Goppa码的高效译码算法[36, 37]。我们引入了扭Goppa 码，通过添加扭项对经典Goppa码进行了推广，并具有高效的译码算法。

QCTM 方案的设计借鉴了Classic McEliece方案[11]和BIG QUAKE方案[1]。与Classic McEliece方案相比，总体公钥尺寸大幅度减少。与BIG QUAKE方案相比，QCTM 方案的公钥尺寸稍微增加，但安全性增强。这样做的期望是取得安全性和效率（特别是公钥尺寸）方面的平衡。总体上，QCTM方案的安全性介于Classic McEliece方案和BIG QUAKE方案之间。

10 结论

QCTM 使用 $m(t-1)+1$ 行扭Goppa 校验结构、局部准循环主体 T_1 和普通尾行 T_2 形成混合公开矩阵。公钥保存为 $(\psi(T_1), T_2)$ ，私钥保存为 (δ, Γ', s) ，解封装时由 Γ' 临时重算译码所需辅助矩阵。该规范与提交实现的三组参数、矩阵形状、公钥大小公式和KEM 接口保持一致。

参考文献

- [1] M. Bardet, E. Barelli, O. Blazy, et al., “BIG QUAKE: BInary Goppa QUAsi-cyclic Key Encapsulation,” NIST Post-Quantum Cryptography Round 1 submission, 2017. NIST Round 1 submissions.
- [2] R. Perlner, “Code based Crypto in the NIST competition: PQC Standardization Process,” NIST presentation slides, May 2019.
- [3] M. Bardet, R. Mora, and J.-P. Tillich, “Polynomial Time Key-Recovery Attack on High Rate Random Alternant Codes,” *IEEE Transactions on Information Theory*, vol. 70, no. 6, pp. 4492–4511, Jun. 2024.
- [4] A. Becker, A. Joux, A. May, and A. Meurer, “Decoding random binary linear codes in $2^{n/20}$: how $1+1=0$ improves information set decoding,” in *Advances in Cryptology – EUROCRYPT 2012*, LNCS 7237, pp. 520–536, Springer, 2012.

- [5] T. P. Berger, “Cyclic alternant codes induced by an automorphism of a GRS code,” in *Finite Field Theory, Applications and Algorithm*, Contemporary Mathematics, vol. 225, pp. 143–154, American Mathematical Society, 1999.
- [6] T. P. Berger, “New classes of cyclic extended Goppa codes,” *IEEE Transactions on Information Theory*, vol. 45, no. 4, pp. 1264–1266, 1999.
- [7] T. P. Berger, “Goppa and related codes invariant under a prescribed permutation,” *IEEE Transactions on Information Theory*, vol. 46, no. 7, pp. 2628–2633, 2000.
- [8] T. P. Berger, “Quasi-cyclic Goppa codes,” in *Proceedings of the IEEE International Symposium on Information Theory*, 2000.
- [9] E. Berlekamp, “Goppa codes,” *IEEE Transactions on Information Theory*, vol. 19, no. 5, pp. 590–592, 1973.
- [10] D. J. Bernstein, T. Lange, and C. Peters, “Smaller decoding exponents: ball-collision decoding,” in *Advances in Cryptology – CRYPTO 2011*, LNCS 6841, pp. 743–760, Springer, 2011.
- [11] D. J. Bernstein, T. Chou, C. Cid, et al., “Classic McEliece: conservative code-based cryptography,” NIST Post-Quantum Cryptography Round 4 submission, 2025. NIST Round 4 submissions.
- [12] L. Both and A. May, “Optimizing BJMM with nearest neighbors: full decoding in $2^{2n/21}$ and McEliece security,” in *Proceedings of the Tenth International Workshop on Coding and Cryptography (WCC 2017)*, 2017.
- [13] A. Couvreur, I. M. Corbella, and R. Pellikaan, “A polynomial time attack against algebraic geometry code based public key cryptosystems,” in *Proceedings of the IEEE International Symposium on Information Theory*, pp. 1446–1450, 2014.
- [14] A. Couvreur, R. Mora, and J.-P. Tillich, “A new approach based on quadratic forms to attack the McEliece cryptosystem,” in J. Guo and R. Steinfield (eds.), *Advances in Cryptology – ASIACRYPT 2023*, LNCS 14441, pp. 3–38, Springer, 2023.
- [15] I. Dumer, “On minimum distance decoding of linear codes,” in *Proceedings of the 5th Joint Soviet-Swedish International Workshop on Information Theory*, Moscow, pp. 50–52, 1991.
- [16] A. Esser and E. Bellini, “Syndrome Decoding Estimator,” in G. Hanaoka et al. (eds.), *Public-Key Cryptography – PKC 2022*, LNCS 13177, pp. 112–141, Springer, 2022.
- [17] J.-C. Faugère, A. Otmani, L. Perret, and J.-P. Tillich, “Algebraic Cryptanalysis of McEliece Variants with Compact Keys,” in H. Gilbert (ed.), *Advances in Cryptology – EUROCRYPT 2010*, LNCS 6110, pp. 279–298, Springer, 2010.
- [18] J.-C. Faugère, L. Perret, and F. de Portzamparc, “Algebraic attack against variants of McEliece with Goppa polynomial of a special form,” in *Advances in Cryptology – ASIACRYPT 2014*, pp. 21–41, Springer, 2014.
- [19] J.-C. Faugère, A. Otmani, L. Perret, F. de Portzamparc, and J.-P. Tillich, “Folding Alternant and Goppa Codes With Non-Trivial Automorphism Groups,” *IEEE Transactions on Information Theory*, vol. 62, no. 1, pp. 184–198, Jan. 2016.
- [20] V. Goppa, “A new class of linear error correcting codes,” *Problems of Information Transmission*, vol. 6, no. 3, pp. 207–212, 1970.
- [21] V. Goppa, “Rational representation of codes and (L, g) codes,” *Problems of Information Transmission*, vol. 7, no. 3, pp. 41–49, 1971.
- [22] D. Huang, Q. Yue, Y. Niu, and X. Li, “MDS or NMDS self-dual codes from twisted generalized Reed–Solomon codes,” *Designs, Codes and Cryptography*, vol. 89, no. 9, pp. 2195–2209, 2021.
- [23] W. Huffman and V. Pless, *Fundamentals of Error-Correcting Codes*. Cambridge University Press, 2003.

- [24] X. Jia, F. Li, H. Sun, Q. Yue, “Generator Polynomials of Cyclic Expurgated or Extended Goppa Codes,” *IEEE Trans. Inf. Theory*, vol. 70, no.11, pp. 7711–7722, 2024.
- [25] F. Li, X. Jia, H. Sun, Q. Yue, “Quasi-cyclic binary extended and expurgated Goppa codes and their parameters,” *Des. Codes Cryptogr.* vol. 94, no. 4, pp. 85, 2026.
- [26] X. Li and Q. Yue, “Non-binary irreducible quasi-cyclic parity-check subcodes of Goppa codes and extended Goppa codes,” *Designs, Codes and Cryptography*, vol. 90, no. 7, pp. 1629–1647, 2022.
- [27] X. Li and Q. Yue, “Construction of expurgated and extended Goppa codes with dihedral automorphism groups,” *IEEE Transactions on Information Theory*, vol. 68, no. 10, pp. 6472–6480, 2022.
- [28] R. Lidl and H. Niederreiter, *Finite Fields*. Cambridge University Press, 2008.
- [29] A. May and I. Ozerov, “On computing nearest neighbors with applications to decoding of binary linear codes,” in *Advances in Cryptology – EUROCRYPT 2015*, LNCS 9056, pp. 203–228, Springer, 2015.
- [30] R. J. McEliece, “A public-key cryptosystem based on algebraic coding theory,” DSN Progress Report 42-44, pp. 114–116, Jet Propulsion Laboratory, 1978.
- [31] R. Misoczki and P. S. L. M. Barreto, “Compact McEliece Keys from Goppa Codes,” in M. J. Jacobson Jr., V. Rijmen, and R. Safavi-Naini (eds.), *Selected Areas in Cryptography – SAC 2009*, LNCS 5867, pp. 376–392, Springer, 2009.
- [32] H. Niederreiter, “Knapsack-type cryptosystems and algebraic coding theory,” *Problems of Control and Information Theory*, vol. 15, no. 2, pp. 159–166, 1986.
- [33] N. J. Patterson, “The algebraic decoding of Goppa codes,” *IEEE Transactions on Information Theory*, vol. 21, no. 2, pp. 203–207, 1975.
- [34] E. Prange, “The use of information sets in decoding cyclic codes,” *IRE Transactions on Information Theory*, vol. 8, no. 5, pp. 5–9, 1962.
- [35] J. Stern, “A method for finding codewords of small weight,” in G. D. Cohen and J. Wolfmann (eds.), *Coding Theory and Applications*, LNCS 388, pp. 106–113, Springer, 1989.
- [36] J. Sui and Q. Yue, “Twisted Goppa codes with an efficient decoding algorithm and quasi-cyclic properties,” *IEEE Transactions on Information Theory*, vol. 69, no. 9, pp. 5660–5669, 2023.
- [37] H. Sun, Q. Yue, X. Jia, and C. Li, “Decoding algorithms of twisted GRS codes and twisted Goppa codes,” *IEEE Transactions on Information Theory*, vol. 71, no. 2, pp. 1018–1027, 2025.
- [38] C. Wieschebrink, “Cryptanalysis of the Niederreiter public key scheme based on GRS subcodes,” in *Post-Quantum Cryptography*, pp. 61–72, Springer, 2010.